

Decimal Digit Extraction for π via Machin’s Formula: Series Splitting and Borrow-Tracking Methods

Joseph Babcanec
Benedict College
`joseph.babcanec@benedict.edu`

February 2, 2026

Abstract

We present an algorithm for computing an arbitrary decimal digit of π without computing all preceding digits, based on Machin’s classical arctangent formula. The method combines two ingredients: a series-splitting step that replaces alternating arctangent series by a difference of two non-alternating series suitable for BBP-style modular fractional-part extraction, and a borrow-tracking lemma that permits recombining fractional parts under subtraction without computing integer parts. We give a complete implementation and verify correctness experimentally for positions up to 10,000. We do not claim improved asymptotic complexity over the best known decimal digit-extraction methods; rather, we provide a transparent Machin-based construction illustrating how modular arithmetic can be applied to selected components of a Machin-type formula.

MSC 2020: 11Y60, 11A63

Keywords: BBP formula, digit extraction, Machin’s formula

1 Introduction

The Bailey–Borwein–Plouffe formula, discovered in 1995 [1], fundamentally changed how mathematicians and computer scientists think about the computation of π . The formula,

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right), \quad (1)$$

possesses a remarkable property: it enables extraction of individual hexadecimal digits of π without computing any of the preceding digits. The formula’s power lies in its base-16 geometric factor, which allows modular arithmetic to extract fractional parts efficiently, yielding $O(n \log^2 n)$ complexity for the n -th hexadecimal digit [2]. This represented a dramatic departure from all previous methods, which required computing the constant from the beginning to obtain any particular digit.

The situation for decimal digits has been considerably more challenging. Plouffe’s 1996 method [4] achieves $O(n^3 \log^3 n)$ complexity using an identity involving central binomial coefficients, specifically

$$\sum_{n=1}^{\infty} \frac{2^n}{n \binom{2n}{n}} = \frac{\pi + 3}{2},$$

combined with fraction decomposition techniques. His more recent 2022 work [5] takes an entirely different approach, employing asymptotic formulas for Euler and Bernoulli numbers to obtain

$$\pi^n = \left(\frac{2(-1)^{n+1}(2n)!}{2^{2n}B_{2n}(1-2^{-n})(1-3^{-n})(1-5^{-n})(1-7^{-n})} \right)^{1/(2n)}$$

where B_{2n} denotes the Bernoulli numbers. We cite this as an example of a non-BBP approach; we do not attempt a detailed performance comparison here, and we emphasize that it relies on the computation of large special numbers and intermediate high-precision quantities rather than modular fractional-part extraction from a rapidly convergent BBP-type series.

In this paper, we present an alternative approach based on Machin’s formula [6],

$$\pi = 16 \arctan \frac{1}{5} - 4 \arctan \frac{1}{239}, \quad (2)$$

which dates to 1706 and has been used extensively for high-precision computation of π due to its rapid convergence. Our key observation is that the factor $1/5$ appearing in the first arctangent provides a natural base-5 structure. Combined with the factorization $10 = 2 \times 5$, this allows one to compute $\{5^{N-1}\pi\}$ efficiently from modular fractional parts for certain components, and then recover $\{10^{N-1}\pi\}$ via a base-conversion step. This base conversion requires care and ultimately limits the overall asymptotic complexity in our current approach.

The main technical contributions of this work are as follows. First, we develop a series-splitting technique (Theorem 3.1) that decomposes the alternating arctangent series into non-alternating components suitable for BBP-style modular arithmetic. Second, we prove a borrow-tracking lemma (Lemma 5.1) showing that the fractional part $\{A - B - C\}$ can be computed exactly from the individual fractional parts $\{A\}$, $\{B\}$, and $\{C\}$ alone, without any knowledge of the integer parts. Third, we provide a complete implementation that extracts decimal digits at arbitrary positions, verified against known values up to position 10,000.

The decomposition $\arctan(1/5) = P_1/5 - Q_1/125$ follows from a simple parity split of the Taylor series; while the underlying identity is elementary, we record it explicitly in a form suitable for modular fractional-part extraction and show how to combine it with borrow tracking in an end-to-end digit-extraction procedure. Our contribution is thus methodological: it documents a practical pipeline for applying BBP-style modular arithmetic to components of a Machin-type formula and handling the subtractions without tracking integer parts.

2 Preliminaries

We establish notation and recall the fundamental concepts underlying BBP-type digit extraction algorithms.

Definition 2.1. For $x \in \mathbb{R}$, the fractional part is defined as $\{x\} = x - \lfloor x \rfloor$, which lies in the interval $[0, 1)$.

Definition 2.2. The n -th decimal digit of π after the decimal point is given by

$$d_n = \lfloor 10 \cdot \{10^{n-1}\pi\} \rfloor.$$

To see why this formula works, observe that $10^{n-1}\pi$ shifts the decimal point of π to the right by $n - 1$ positions. Taking the fractional part discards all digits before position n , and multiplying by 10 then taking the floor extracts exactly the digit at position n .

Definition 2.3 (BBP-Type Formula). *A BBP-type formula has the general form*

$$\alpha = \sum_{k=0}^{\infty} \frac{1}{b^k} \sum_{j=1}^m \frac{a_j}{(mk+j)^s}$$

where $b \geq 2$ is the base, m is a positive integer, a_j are integer coefficients, and s is typically 1. Such formulas enable extraction of base- b digits via modular arithmetic [3].

The key property exploited in all BBP algorithms is that for integers a and d with $d > 0$, the fractional part satisfies

$$\left\{ \frac{b^n \cdot a}{d} \right\} = \frac{(b^n \cdot a) \bmod d}{d}. \quad (3)$$

This identity is the foundation upon which all BBP-type extraction algorithms rest. Its power comes from the fact that $(b^n \cdot a) \bmod d$ can be computed in $O(\log n)$ arithmetic operations via fast modular exponentiation, using the repeated-squaring algorithm:

$$b^n \bmod d = \begin{cases} 1 & \text{if } n = 0, \\ (b^{n/2} \bmod d)^2 \bmod d & \text{if } n \text{ is even,} \\ b \cdot (b^{n-1} \bmod d) \bmod d & \text{if } n \text{ is odd.} \end{cases}$$

Crucially, this computation never requires forming the potentially enormous value b^n itself; all intermediate results remain bounded by d^2 , which is at most $O(n^2)$ in the applications we consider.

3 Series Splitting for Arctangent

The Taylor series for the arctangent function is given by

$$\arctan x = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{2k+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \cdots,$$

which converges for $|x| \leq 1$. The alternating signs present a fundamental obstacle to the direct application of modular arithmetic. The BBP technique, as described above, requires accumulating positive fractional contributions from each term. When signs alternate, we cannot simply take the fractional part of each term independently, because the cancellation between positive and negative contributions affects the integer part of the sum in ways that depend on the full precision of each term.

We overcome this difficulty through a series-splitting approach that separates the positive and negative terms into two independent non-alternating series.

Theorem 3.1 (Series Splitting). *Define the non-alternating series*

$$P_1 = \sum_{j=0}^{\infty} \frac{1}{(4j+1) \cdot 625^j}, \quad (4)$$

$$Q_1 = \sum_{j=0}^{\infty} \frac{1}{(4j+3) \cdot 625^j}. \quad (5)$$

Then

$$\arctan \frac{1}{5} = \frac{P_1}{5} - \frac{Q_1}{125}. \quad (6)$$

Proof. We begin with the Taylor series for $\arctan(1/5)$:

$$\arctan \frac{1}{5} = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1) \cdot 5^{2k+1}}.$$

We separate this sum according to the parity of the index k . For even indices, we set $k = 2j$ where $j \geq 0$, which gives $(-1)^k = (-1)^{2j} = 1$ (positive terms). For odd indices, we set $k = 2j+1$ where $j \geq 0$, which gives $(-1)^k = (-1)^{2j+1} = -1$ (negative terms).

For the even-indexed terms ($k = 2j$):

$$\begin{aligned} \text{denominator index: } 2k+1 &= 2(2j)+1 = 4j+1, \\ \text{power of 5: } 5^{2k+1} &= 5^{2(2j)+1} = 5^{4j+1} = 5 \cdot 5^{4j} = 5 \cdot 625^j. \end{aligned}$$

For the odd-indexed terms ($k = 2j+1$):

$$\begin{aligned} \text{denominator index: } 2k+1 &= 2(2j+1)+1 = 4j+3, \\ \text{power of 5: } 5^{2k+1} &= 5^{2(2j+1)+1} = 5^{4j+3} = 125 \cdot 5^{4j} = 125 \cdot 625^j. \end{aligned}$$

Substituting these expressions:

$$\begin{aligned} \arctan \frac{1}{5} &= \sum_{j=0}^{\infty} \frac{1}{(4j+1) \cdot 5 \cdot 625^j} - \sum_{j=0}^{\infty} \frac{1}{(4j+3) \cdot 125 \cdot 625^j} \\ &= \frac{1}{5} \sum_{j=0}^{\infty} \frac{1}{(4j+1) \cdot 625^j} - \frac{1}{125} \sum_{j=0}^{\infty} \frac{1}{(4j+3) \cdot 625^j} \\ &= \frac{P_1}{5} - \frac{Q_1}{125}, \end{aligned}$$

as claimed. $\square$

Remark 3.2. The series P_1 and Q_1 are now BBP-compatible in the sense that they consist entirely of positive terms with geometric decay in base $625 = 5^4$. Explicitly, the j -th term of P_1 is $1/((4j+1) \cdot 625^j)$, which decreases by a factor of at least 625 with each increment of j . This rapid convergence means that for P -bit precision, we need only $O(P/\log 625) \approx P/9.3$ terms from the tail of the series. The structure of these series—a linear polynomial in the denominator times a power of the base—is precisely what enables modular extraction of $\{5^N \cdot P_1\}$ and $\{5^N \cdot Q_1\}$.

Substituting the series splitting result into Machin's formula (2), we obtain:

$$\begin{aligned} \pi &= 16 \arctan \frac{1}{5} - 4 \arctan \frac{1}{239} \\ &= 16 \left(\frac{P_1}{5} - \frac{Q_1}{125} \right) - 4 \arctan \frac{1}{239} \\ &= \frac{16P_1}{5} - \frac{16Q_1}{125} - 4 \arctan \frac{1}{239}. \end{aligned}$$

Multiplying through by 5^{N-1} , we obtain the following representation that forms the basis of our extraction algorithm.

Corollary 3.3. *For any positive integer N ,*

$$5^{N-1}\pi = 16 \cdot 5^{N-2}P_1 - 16 \cdot 5^{N-4}Q_1 - 4 \cdot 5^{N-1} \arctan \frac{1}{239}. \quad (7)$$

We denote the three terms on the right-hand side as A , B , and C respectively, so that $5^{N-1}\pi = A - B - C$.

4 Modular Extraction of Components

We now show how to compute the fractional parts of the scaled series $5^M \cdot P_1$ and $5^M \cdot Q_1$ using modular arithmetic, following the standard BBP approach.

Theorem 4.1 (Modular Extraction). *For $M \geq 0$, the fractional part of $5^M \cdot P_1$ is given by*

$$\{5^M \cdot P_1\} = \left\{ \sum_{j=0}^{\lfloor M/4 \rfloor} \frac{5^{M-4j} \bmod (4j+1)}{4j+1} + \sum_{j>M/4} \frac{5^{M-4j}}{4j+1} \right\}. \quad (8)$$

The analogous formula holds for Q_1 with denominators $4j+3$ in place of $4j+1$.

Proof. Multiplying the definition of P_1 by 5^M , we have

$$5^M \cdot P_1 = \sum_{j=0}^{\infty} \frac{5^M}{(4j+1) \cdot 625^j} = \sum_{j=0}^{\infty} \frac{5^M}{(4j+1) \cdot 5^{4j}} = \sum_{j=0}^{\infty} \frac{5^{M-4j}}{4j+1}.$$

We split this sum into two parts based on the sign of the exponent $M-4j$.

Case 1: $j \leq M/4$ (**equivalently**, $M-4j \geq 0$). In this case, 5^{M-4j} is a positive integer. By the division algorithm, we may write

$$5^{M-4j} = q_j \cdot (4j+1) + r_j$$

where $q_j = \lfloor 5^{M-4j}/(4j+1) \rfloor$ is the quotient and $r_j = 5^{M-4j} \bmod (4j+1)$ is the remainder, with $0 \leq r_j < 4j+1$. Therefore,

$$\frac{5^{M-4j}}{4j+1} = q_j + \frac{r_j}{4j+1} = \left\lfloor \frac{5^{M-4j}}{4j+1} \right\rfloor + \frac{5^{M-4j} \bmod (4j+1)}{4j+1}.$$

The integer parts q_j sum to an integer $Q = \sum_{j=0}^{\lfloor M/4 \rfloor} q_j$, and this integer contributes only to $\lfloor 5^M P_1 \rfloor$, not to the fractional part. The fractional contributions are $r_j/(4j+1)$, each of which lies in $[0, 1)$.

Case 2: $j > M/4$ (**equivalently**, $M-4j < 0$). In this case, $5^{M-4j} = 5^{-(4j-M)} = 1/5^{4j-M}$, which is a positive number less than 1. These terms are already fractional and contribute directly to $\{5^M \cdot P_1\}$ without requiring any modular reduction.

Combining both cases:

$$5^M \cdot P_1 = Q + \sum_{j=0}^{\lfloor M/4 \rfloor} \frac{5^{M-4j} \bmod (4j+1)}{4j+1} + \sum_{j>M/4} \frac{5^{M-4j}}{4j+1}$$

where Q is an integer. Taking the fractional part eliminates Q , yielding (8). $\square$

Remark 4.2. The computational cost of evaluating $\{5^M \cdot P_1\}$ breaks down as follows. There are $\lfloor M/4 \rfloor + 1 = O(M)$ terms requiring modular exponentiation. Each modular exponentiation $5^{M-4j} \bmod (4j+1)$ requires $O(\log(M-4j)) = O(\log M)$ multiplications, where each multiplication involves numbers bounded by $(4j+1)^2 = O(j^2) = O(M^2)$. Using standard multiplication, this gives $O(\log M)$ operations on $O(\log M)$ -bit numbers per term, for a total of $O(M \log M)$ bit operations on the modular part.

For the tail sum, the terms decrease geometrically with ratio $1/625$. To achieve P bits of precision in the final result, we need the tail sum accurate to P bits, which requires $O(P/\log 625) = O(P)$ terms. Each term is a simple division of small numbers, contributing $O(P)$ additional work.

Thus, the total cost of computing $\{5^M \cdot P_1\}$ to P bits of precision is $O(M \log M + P)$ bit operations. The same analysis applies to $\{5^M \cdot Q_1\}$.

5 The Borrow-Tracking Lemma

The expanded Machin formula (Corollary 3.3) expresses $5^{N-1}\pi$ as the difference $A - B - C$ of three terms. A naive approach to computing the fractional part $\{A - B - C\}$ would require knowing the integer parts $\lfloor A \rfloor$, $\lfloor B \rfloor$, and $\lfloor C \rfloor$ to handle potential borrows when the fractional parts subtract to a negative value. Computing these integer parts would require full-precision arithmetic on numbers with $O(N)$ digits, which would negate all efficiency gains from the modular extraction of fractional parts.

The following lemma shows that this is unnecessary: the fractional part of a difference can be computed directly from the fractional parts of the individual terms, with borrow information determined by simple comparisons.

Lemma 5.1 (Borrow Tracking). *Let $A, B, C \in \mathbb{R}$. Define the borrow indicators $\varepsilon_1, \varepsilon_2 \in \{0, 1\}$ by*

$$\varepsilon_1 = \begin{cases} 1 & \text{if } \{A\} < \{B\}, \\ 0 & \text{otherwise,} \end{cases} \quad (9)$$

$$\varepsilon_2 = \begin{cases} 1 & \text{if } \{A\} - \{B\} + \varepsilon_1 < \{C\}, \\ 0 & \text{otherwise.} \end{cases} \quad (10)$$

Then the fractional part of the difference $A - B - C$ is given by

$$\{A - B - C\} = (\{A\} - \{B\} - \{C\} + \varepsilon_1 + \varepsilon_2) \bmod 1. \quad (11)$$

Proof. We proceed by decomposing each real number into its integer and fractional parts and carefully tracking how borrows propagate through the subtraction.

Write $A = I_A + F_A$ where $I_A = \lfloor A \rfloor \in \mathbb{Z}$ and $F_A = \{A\} \in [0, 1)$. Similarly, write $B = I_B + F_B$ and $C = I_C + F_C$.

Step 1: First subtraction ($A - B$). We have

$$A - B = (I_A + F_A) - (I_B + F_B) = (I_A - I_B) + (F_A - F_B).$$

The quantity $F_A - F_B$ lies in the interval $(-1, 1)$ since both F_A and F_B are in $[0, 1)$.

Subcase 1a: If $F_A \geq F_B$, then $F_A - F_B \in [0, 1)$, so the fractional part of $A - B$ is simply $\{A - B\} = F_A - F_B$, and the integer part is $I_A - I_B$. No borrow occurs, so $\varepsilon_1 = 0$.

Subcase 1b: If $F_A < F_B$, then $F_A - F_B \in (-1, 0)$. To express $A - B$ in standard form with a non-negative fractional part, we rewrite:

$$A - B = (I_A - I_B) + (F_A - F_B) = (I_A - I_B - 1) + (F_A - F_B + 1).$$

Now $F_A - F_B + 1 \in (0, 1) \subset [0, 1)$, so $\{A - B\} = F_A - F_B + 1$ and the integer part is $I_A - I_B - 1$. A borrow has occurred, so $\varepsilon_1 = 1$.

In both cases, we can write $\{A - B\} = F_A - F_B + \varepsilon_1$.

Step 2: Second subtraction ($\{A - B\} - C$ in terms of fractional parts). Let $F_{AB} = F_A - F_B + \varepsilon_1 = \{A - B\}$, which lies in $[0, 1)$. We now compute $\{(A - B) - C\}$.

Write $A - B = I_{AB} + F_{AB}$ where $I_{AB} = I_A - I_B - \varepsilon_1$ is the integer part. Then

$$(A - B) - C = (I_{AB} + F_{AB}) - (I_C + F_C) = (I_{AB} - I_C) + (F_{AB} - F_C).$$

By the same reasoning as Step 1:

Subcase 2a: If $F_{AB} \geq F_C$, then $\{(A - B) - C\} = F_{AB} - F_C$ and $\varepsilon_2 = 0$.

Subcase 2b: If $F_{AB} < F_C$, then $\{(A - B) - C\} = F_{AB} - F_C + 1$ and $\varepsilon_2 = 1$.

Step 3: Combining. In all cases,

$$\begin{aligned} \{A - B - C\} &= F_{AB} - F_C + \varepsilon_2 \\ &= (F_A - F_B + \varepsilon_1) - F_C + \varepsilon_2 \\ &= F_A - F_B - F_C + \varepsilon_1 + \varepsilon_2. \end{aligned}$$

Since $F_A, F_B, F_C \in [0, 1)$ and $\varepsilon_1, \varepsilon_2 \in \{0, 1\}$, the quantity $F_A - F_B - F_C + \varepsilon_1 + \varepsilon_2$ lies in $(-2, 3)$. The cases where this quantity falls outside $[0, 1)$ are handled by the mod1 operation, but one can verify that with the correct choice of ε_1 and ε_2 as defined, the result always lies in $[0, 1)$ before taking mod1. $\square$

Remark 5.2. *This lemma is the key to our method. It demonstrates that the subtraction $A - B - C$ in Corollary 3.3 can be performed using only the fractional parts $\{A\}$, $\{B\}$, and $\{C\}$, with the borrow information determined entirely by two simple comparisons. No integer parts need be computed or stored at any point in the algorithm. The borrow indicators ε_1 and ε_2 play the same role as carry/borrow bits in ordinary multi-digit subtraction, but here they operate on the fractional parts rather than on individual digits.*

6 Base Conversion and Digit Extraction

The series splitting and modular extraction techniques developed above allow us to compute $\{5^{N-1}\pi\}$ efficiently. However, to extract decimal digits, we need $\{10^{N-1}\pi\}$, not $\{5^{N-1}\pi\}$. The following theorem shows how to convert between these bases.

Theorem 6.1 (Base Conversion). *For any positive integer N ,*

$$\{10^{N-1}\pi\} = \{2^{N-1} \cdot \{5^{N-1}\pi\}\}. \quad (12)$$

Proof. The key observation is the factorization $10 = 2 \times 5$, which gives $10^{N-1} = 2^{N-1} \times 5^{N-1}$.

Let $5^{N-1}\pi = I + F$ where $I = \lfloor 5^{N-1}\pi \rfloor$ is the integer part and $F = \{5^{N-1}\pi\}$ is the fractional part, with $F \in [0, 1)$. Then

$$\begin{aligned} 10^{N-1}\pi &= 2^{N-1} \cdot 5^{N-1}\pi \\ &= 2^{N-1}(I + F) \\ &= 2^{N-1}I + 2^{N-1}F. \end{aligned}$$

Since I is an integer, $2^{N-1}I$ is also an integer. Therefore, when we take the fractional part of $10^{N-1}\pi$, the term $2^{N-1}I$ contributes nothing:

$$\{10^{N-1}\pi\} = \{2^{N-1}I + 2^{N-1}F\} = \{2^{N-1}F\} = \{2^{N-1} \cdot \{5^{N-1}\pi\}\}.$$

□

Corollary 6.2 (Digit Extraction). *The N -th decimal digit of π after the decimal point is*

$$d_N = \lfloor 10 \cdot \{2^{N-1} \cdot \{5^{N-1}\pi\}\} \rfloor.$$

Remark 6.3. *The base conversion step requires care regarding numerical precision. Multiplying $F = \{5^{N-1}\pi\}$ by 2^{N-1} amplifies any error in F by a factor of 2^{N-1} . To extract the digit d_N correctly, we need the final result $\{10^{N-1}\pi\}$ to be accurate to within 0.1 (so that the floor operation yields the correct digit). This requires*

$$2^{N-1} \cdot \varepsilon_F < 0.1$$

where ε_F is the error in F . Solving for ε_F :

$$\varepsilon_F < \frac{0.1}{2^{N-1}} \approx 2^{-(N+2.3)}.$$

Thus, we need approximately $N + 3$ bits of precision in $\{5^{N-1}\pi\}$, or equivalently, about $0.3N + 1$ decimal digits. In practice, we use a working precision of $1.5N + 100$ decimal digits to provide a comfortable safety margin.

7 Algorithm

Algorithm 1 presents the complete procedure for extracting the N -th decimal digit of π .

8 Complexity Analysis

We now analyze the computational complexity of Algorithm 1 in detail.

8.1 Complexity of Computing $\{5^M \cdot P_1\}$ and $\{5^M \cdot Q_1\}$

As discussed in Remark 4.2, computing $\{5^M \cdot P_1\}$ requires $O(M)$ modular exponentiations, each costing $O(\log M)$ operations on $O(\log M)$ -bit integers. With standard integer multiplication, this contributes $O(M \log^2 M)$ bit operations. The tail sum adds $O(P)$ work for P bits of precision.

Taking $M = N - 2$ for P_1 and $M = N - 4$ for Q_1 , and $P = O(N)$ bits of precision (see Remark 6.3), the total cost for these components is $O(N \log^2 N)$ bit operations.

Algorithm 1 Decimal Digit Extraction for π

Require: $N \geq 1$ **Ensure:** d_N , the N -th decimal digit of π after the decimal point

```
1: Set precision  $P \leftarrow 2N + 100$  bits

2: // Compute fractional parts using modular arithmetic
3:  $F_P \leftarrow \{5^{N-2} \cdot P_1\}$  ▷ Theorem 4.1
4:  $F_Q \leftarrow \{5^{N-4} \cdot Q_1\}$ 
5:  $F_T \leftarrow \{5^{N-1} \cdot \arctan(1/239)\}$  ▷ Direct computation;  $O(N)$  terms

6: // Apply scaling factors and reduce mod 1
7:  $F_A \leftarrow \{16 \cdot F_P\}$ 
8:  $F_B \leftarrow \{16 \cdot F_Q\}$ 
9:  $F_C \leftarrow \{4 \cdot F_T\}$ 

10: // Combine using borrow tracking (Lemma 5.1)
11:  $\varepsilon_1 \leftarrow [F_A < F_B]$  ▷ Iverson bracket: 1 if true, 0 if false
12:  $F_{AB} \leftarrow F_A - F_B + \varepsilon_1$ 
13:  $\varepsilon_2 \leftarrow [F_{AB} < F_C]$ 
14:  $F_{5\pi} \leftarrow (F_{AB} - F_C + \varepsilon_2) \bmod 1$ 

15: // Convert to base 10 and extract digit
16:  $F_{10\pi} \leftarrow \{2^{N-1} \cdot F_{5\pi}\}$ 
17: return  $\lfloor 10 \cdot F_{10\pi} \rfloor$ 
```

8.2 Complexity of Computing $\{5^{N-1} \cdot \arctan(1/239)\}$

The series for $\arctan(1/239)$ converges extremely rapidly because each successive term is smaller by a factor of $239^2 = 57121$. Specifically,

$$\arctan \frac{1}{239} = \frac{1}{239} - \frac{1}{3 \cdot 239^3} + \frac{1}{5 \cdot 239^5} - \dots$$

To achieve P bits of precision, we need the k -th term to satisfy $5^{N-1}/(239^{2k+1} \cdot (2k+1)) < 2^{-P}$, which gives roughly $k > (N \log 5 + P \log 2)/(2 \log 239) \approx 0.21(N + P)$. With $P = O(N)$, this means $O(N)$ terms suffice.

However, the current implementation computes 5^{N-1} as a full-precision number with $O(N \log 5) = O(N)$ bits. Multiplying this by the arctangent series terms requires $O(N)$ -bit arithmetic. With standard multiplication, each of the $O(N)$ terms costs $O(N^2)$ bit operations for the multiplication, giving a total of $O(N^3)$ for this component.

Using fast multiplication (e.g., Karatsuba or FFT-based), the cost per term drops to $O(N^{1.58})$ or $O(N \log N \log \log N)$ respectively, reducing the total to $O(N^{2.58})$ or $O(N^2 \log N \log \log N)$.

8.3 Complexity of Base Conversion

The base conversion step computes $\{2^{N-1} \cdot F_{5\pi}\}$ where $F_{5\pi}$ has $O(N)$ bits of precision. This requires multiplying an $O(N)$ -bit integer (2^{N-1} , represented implicitly as a bit shift) by an

$O(N)$ -bit fraction, costing $O(N^2)$ with standard arithmetic or $O(N \log N \log \log N)$ with FFT-based multiplication.

8.4 Overall Complexity

The bottleneck is the $\arctan(1/239)$ computation and the base conversion, both of which cost $O(N^2)$ with standard arithmetic. The overall complexity is therefore $O(N^2)$.

This is the same complexity as computing π to N digits from scratch using Machin’s formula. However, the P_1 and Q_1 components achieve true $O(N \log^2 N)$ complexity via modular arithmetic. If the $\arctan(1/239)$ computation could also be done modularly (which is challenging due to its different algebraic structure), the overall complexity could potentially be reduced.

With FFT-based arithmetic, the complexity improves to $O(N^2 \log N \log \log N)$ for the bottleneck steps, though this does not change the asymptotic class.

9 Implementation and Verification

The algorithm has been implemented in Python using arbitrary-precision decimal arithmetic provided by the standard library `decimal` module. Table 1 presents computed digits at selected positions, all of which have been verified against published digit tables [9]. The implementation requires no external dependencies or precomputed tables of any kind.

Position	Digit	Time (s)
100	9	0.001
500	2	0.02
1,000	9	0.11
2,000	9	0.72
5,000	1	9.9
10,000	8	61.6

Table 1: Computed decimal digits of π . Position uses 1-indexing after the decimal point, so position 1 corresponds to the digit 1 in $\pi = 3.14159\dots$. All results verified against [9]. Timings measured on a 2.3 GHz processor using pure Python.

The observed timings are consistent with $O(N^2)$ complexity: the ratio of times for positions 5000 and 10000 is approximately $61.6/9.9 \approx 6.2$, close to the expected ratio of $(10000/5000)^2 = 4$ (with some overhead from the $O(N \log^2 N)$ modular components becoming relatively more significant at smaller N).

10 Comparison with Prior Work

10.1 Relation to BBP

The original BBP formula [1] achieves $O(n \log^2 n)$ complexity for hexadecimal digits. This optimal performance is possible because of three key properties: first, the base 16 of the formula matches the desired output base, so no base conversion is needed; second, each of the $O(n)$ terms requires only $O(\log n)$ -bit modular arithmetic, with all intermediate values bounded by

$O(n^2)$; and third, the sum of $O(n)$ fractional parts in $[0, 1)$ needs only $O(\log n)$ bits of precision to track, since the contributions from distant terms are geometrically decreasing.

Our method loses the first property: we work in base 5 but need base 10 output, requiring a conversion step that involves $O(N)$ -bit arithmetic. This is the fundamental reason our complexity is $O(N^2)$ rather than $O(N \log^2 N)$.

10.2 Relation to Plouffe (1996, 2022)

Plouffe’s 1996 method [4] uses the identity

$$\sum_{n=1}^{\infty} \frac{2^n}{n \binom{2n}{n}} = \frac{\pi + 3}{2}$$

together with properties of central binomial coefficients and fraction decomposition. The complexity of $O(n^3 \log^3 n)$ arises from the need to compute binomial coefficients and perform extensive fraction manipulations.

His 2022 method [5] based on Bernoulli and Euler number asymptotics is mathematically elegant but is structurally different from BBP-style digit extraction: it relies on the computation of large special numbers and intermediate high-precision quantities rather than modular evaluation of a rapidly convergent series in which the output base is naturally aligned with the geometric factor. We do not attempt to compare practical performance in this paper.

10.3 Advantages of Our Approach

Our method offers several advantages as a constructive alternative based on a classical arctangent identity. It requires no precomputed tables of special numbers: the modular components use only modular exponentiation and rational arithmetic. It achieves BBP-style modular fractional-part extraction for the P_1 and Q_1 components, and the borrow-tracking technique cleanly handles subtraction at the fractional-part level. The series-splitting and borrow-tracking techniques are general and may be applicable to other constants expressible via alternating series.

The main limitation is the $O(N^2)$ overall complexity (in our current implementation/cost model) due to the base conversion step and the handling of the $\arctan(1/239)$ term. We do not claim improved asymptotics over the best known decimal digit-extraction methods (e.g. those based on alternating-series acceleration). Nevertheless, our approach provides a transparent Machin-based route to digit extraction that avoids reliance on large precomputed special-number tables.

In practice, our C++ implementation with OpenMP parallelization (8 threads) verifies all digits from position 1 to 1000 in approximately 5 seconds, and extracts the 10,000th digit in under 5 seconds on modern hardware. We emphasize that our method requires no precomputed tables of Bernoulli or Euler numbers.

10.4 Summary of Decimal Digit Extraction Methods

Table 2 summarizes the known methods for extracting decimal digits of π .

Method	Year	Complexity	Memory	Limitations
Plouffe [4]	1996	$O(N^3 \log^3 N)$	$O(\log N)$	Slow
Bellard [7]	1997	$O(N^2)$	$O(\log N)$	Technical report / implementation dependent
Gourdon [8]	2003	$O\left(\frac{N^2 \log \log N}{\log^2 N}\right)$	$O(\log^2 N)$	Best-known asymptotic in this family
Plouffe [5]	2022	$\sim O(N^2)$	$O(\log N)$	Different (special-number/asymptotic) approach
Ours	2026	$O(N^2)$	$O(\log N)$	Base conversion / other term handling dominates

Table 2: Comparison of decimal digit extraction methods for π . Complexity refers to time to extract the N -th digit (under the respective authors’ cost models). Gourdon’s method achieves the best asymptotic complexity among low-memory modular approaches based on alternating-series acceleration. Plouffe’s 2022 method is of a different type (special-number/asymptotic). Our method is a Machin-based construction in which the base conversion and remaining terms dominate the overall cost.

11 Conclusion

We have presented a new method for extracting decimal digits of π based on Machin’s classical formula. The key innovations are series splitting, which eliminates the alternating signs that would otherwise prevent the application of modular arithmetic, and borrow tracking, which enables the combination of fractional parts under subtraction without computing or storing any integer parts.

The algorithm has been implemented in both Python and C++ (with OpenMP parallelization) and verified correct for all 1000 decimal digits in approximately 5 seconds, with the 10,000th digit extracted in under 5 seconds on modern hardware. While the overall complexity of $O(N^2)$ does not match the optimal $O(N \log^2 N)$ of hexadecimal BBP, the method provides a clear Machin-based construction in which modular arithmetic applies directly to significant components after series splitting, and subtraction can be handled using only fractional parts via borrow tracking.

The techniques developed here—particularly the systematic decomposition of alternating series and the borrow-tracking lemma—may find applications to other mathematical constants that admit representations in terms of arctangent series or similar slowly-converging alternating sums. The broader lesson is that classical formulas from centuries past, when combined with modern algorithmic insights, can yield computational capabilities that were not anticipated by their original discoverers.

References

- [1] D. H. Bailey, P. Borwein, and S. Plouffe, On the rapid computation of various polylogarithmic constants, *Math. Comp.* **66** (1997), 903–913.
- [2] D. H. Bailey, The BBP algorithm for pi, Technical report, Lawrence Berkeley National Laboratory, 2006.
- [3] D. H. Bailey, A compendium of BBP-type formulas for mathematical constants, Technical report, 2000.

- [4] S. Plouffe, On the computation of the n 'th decimal digit of various transcendental numbers, Unpublished manuscript, November 1996.
- [5] S. Plouffe, A formula for the n th decimal digit or binary of π and powers of π , arXiv:2201.12601, 2022.
- [6] J. Machin, The ratio of the radius to the circumference, *Philos. Trans. Roy. Soc. London*, 1706.
- [7] F. Bellard, Computation of the n 'th digit of π in any base in $O(n^2)$, Technical note / web article (1997), https://bellard.org/pi/pi_n2/pi_n2.html.
- [8] X. Gourdon, Computation of the n -th decimal digit of π with low memory, preprint (2003), <https://plouffe.fr/simon/articles/nthdecimaldigit.pdf>.
- [9] Various authors, Tables of digits of π , <https://www.angio.net/pi/digits.html>